



Resumo Analítico de Segurança Informática

Organizado por Matéria | Baseado nas Aulas 01-20 do ISEL (DEETC)



1. Fundamentos de Criptografia

1.1 Conceitos Básicos

Objetivos da Segurança Informática (CIA Triad)

Conceito	Definição	Exemplo
Confidencialidade	Garantir que a informação só é acessível a entidades autorizadas.	Cifrar mensagens com AES.
Integridade	Garantir que a informação não é alterada de forma não autorizada.	Usar <i>hashes</i> (SHA-256) ou HMAC para detetar alterações.
Disponibilidade	Garantir que a informação está disponível quando necessário.	Proteção contra ataques DDoS.
Autenticidade	Garantir que a informação vem de uma fonte confiável.	Assinaturas digitais (RSA).
Não-repúdio	Garantir que uma entidade não pode negar uma ação (ex: envio de uma mensagem).	Assinaturas digitais.

Classificação de Sistemas Criptográficos

Tipo	Descrição	Exemplos	Vantagens	Desvantagens
Simétrico	Usa a mesma chave para cifrar e decifrar.	AES, DES, ChaCha20	Rápido, eficiente para grandes dados.	Problema de distribuição da chave.
Assimétrico	Usa um par de chaves (pública/privada).	RSA, ECC, Diffie-Hellman	Resolve distribuição de chaves.	Lento, complexo.
Hash	Função one-way que mapeia uma entrada de tamanho arbitrário para uma saída de tamanho fixo.	SHA-256, SHA-3, BLAKE2	Integridade, não-repúdio.	Não é cifra (não reversível).

Princípios de Segurança

- Princípio de Kerckhoffs:

"A segurança de um sistema criptográfico deve depender apenas do segredo da chave, não do segredo do algoritmo."

- Exemplo: AES é público, mas seguro porque a chave é secreta.
- **Modelo de Ameaças:**
 - **Ameaça:** Evento potencial que pode causar dano (ex: atacante).
 - **Vulnerabilidade:** Fraqueza no sistema que pode ser explorada (ex: *buffer overflow*).
 - **Ataque:** Exploração ativa de uma vulnerabilidade (ex: *brute force*).

1.2 Cifras Simétricas

Cifras de Bloco

Cifra	Tamanho do Bloco	Tamanho da Chave	Rondas	Estado Atual	Notas
AES	128 bits	128/192/256 bits	10/12/14	Seguro	Standard atual (FIPS 197). Usado em TLS, Wi-Fi, etc.
DES	64 bits	56 bits	16	Obsoleto	Vulnerável a <i>brute force</i> (256 combinações).
3DES	64 bits	112/168 bits	48	Obsoleto	Vulnerável a <i>Sweet32</i> (ataque a blocos de 64 bits).
Blowfish	64 bits	32-448 bits	16	Seguro	Usado em <i>password hashing</i> (ex: <i>bcrypt</i>).

- **Funcionamento do AES:**
 - **Passos por Ronda:**
 1. **SubBytes:** Substituição de bytes usando uma *S-box*.
 2. **ShiftRows:** Deslocamento cíclico de linhas.
 3. **MixColumns:** Multiplicação matricial sobre GF(28).
 4. **AddRoundKey:** XOR com a subchave.
 - **Última Ronda:** Não tem **MixColumns**.
 - **Número de Rondas:**
 - 128 bits: 10 rondas.
 - 192 bits: 12 rondas.
 - 256 bits: 14 rondas.
- **Exemplo (OpenSSL):**

```
# Cifrar com AES-256-CBC
openssl enc -aes-256-cbc -salt -in plaintext.txt -out ciphertext.bin -K
00112233445566778899AABBCCDDEEFF -iv 0123456789ABCDEF0123456789ABCDEF

# Decifrar
openssl enc -d -aes-256-cbc -in ciphertext.bin -out plaintext_recovered.txt -K
00112233445566778899AABBCCDDEEFF -iv 0123456789ABCDEF0123456789ABCDEF
```

Cifras de Fluxo

Cifra	Descrição	Estado Atual	Notas
RC4	Cifra de fluxo simples.	Obsoleto	Vulnerável a ataques de <i>bias</i> (ex: WEP).
ChaCha20	Cifra de fluxo moderna (usada em TLS 1.3).	Seguro	Alternativa ao AES em dispositivos com recursos limitados (ex: <i>mobile</i>).
Salsa20	Predecessor do ChaCha20.	Seguro	Menos usado que ChaCha20.

- **Funcionamento:**
 - Gera um **keystream** (sequência de bits pseudo-aleatória) usando uma chave e um *nonce*.
 - Cifra/decifra com **XOR** entre o *keystream* e o texto plano/criptograma.
 - Fórmula:

$$c_i = m_i \oplus \text{keystream}_i$$
$$m_i = c_i \oplus \text{keystream}_i$$

1.3 Modos de Operação

Visão Geral

Modo	Descrição	Vantagens	Desvantagens	Uso Típico
ECB	Cifra cada bloco independentemente .	Simples, rápido, acesso aleatório, paralelizável.	Não esconde padrões (blocos iguais → criptogramas iguais).	Nunca para dados com padrões (ex: texto, imagens).
CBC	Cada bloco é XORado com o criptograma do bloco anterior antes de ser cifrado.	Esconde padrões.	Propagação de erros (2 blocos), não paralelizável, não acesso aleatório.	Ficheiros, bases de dados.
CTR	Usa um contador para gerar o <i>keystream</i> (usando cifra de bloco).	Acesso aleatório, paralelizável, não propagação de erros.	Requer <i>nonce</i> único.	<i>Streaming</i> , TLS.
GCM	Combina CTR (confidencialidade) + GMAC (integridade).	Confidencialidade + integridade, acesso aleatório, paralelizável.	Complexo.	TLS 1.3, IPsec.
OFB	<i>Keystream</i> é gerado independentemente do texto plano.	Não propagação de erros.	Um bit errado no <i>keystream</i> afeta todos os blocos seguintes.	<i>Streaming</i> .
CFB	Similar ao CBC, mas usa a cifra para gerar o <i>keystream</i> .	Esconde padrões.	Propagação de erros (2 blocos).	<i>Streaming</i> .

Detalhes dos Modos

ECB (Electronic Codebook)

- **Fórmula:**

$$\begin{aligned}c_i &= E(k, m_i) \\ m_i &= D(k, c_i)\end{aligned}$$

- **Problema:**

- Blocos iguais no texto plano → **blocos iguais no criptograma**.
- Exemplo: Cifrar uma imagem com ECB revela contornos (ver [exemplo visual](#)).

CBC (Cipher Block Chaining)

- **Fórmula:**

$$\begin{aligned}c_i &= E(k, m_i \oplus c_{i-1}) \\ m_i &= D(k, c_i) \oplus c_{i-1}\end{aligned}$$

- $c_0 = IV$ (Vetor de Inicialização).

- **Requisitos do IV:**

- **Único** por mensagem (evita repetições).
- **Imprevisível** (evita ataques de *chosen plaintext*).
- **Não precisa ser secreto** (pode ser enviado em texto plano).

CTR (Counter)

- **Fórmula:**

$$\begin{aligned}c_i &= m_i \oplus E(k, \text{contador}) \\ m_i &= c_i \oplus E(k, \text{contador})\end{aligned}$$

- $\text{contador} = \text{nonce} || i$ (onde i é o número do bloco).

- **Vantagens:**

- **Acesso aleatório:** Decifra qualquer bloco isoladamente.
- **Paralelizável:** Blocos podem ser cifrados/decifrados em paralelo.
- **IV pode ser público** (mas deve ser único).

GCM (Galois/Counter Mode)

- **Combina:**

- **CTR** para **confidencialidade**.
- **GMAC** para **integridade/autenticidade**.

- **Fórmula:**

- Cifra: $c = \text{CTR}(k, m) || \text{GMAC}(k, m, IV)$.
- Decifra: Verifica o GMAC antes de decifrar.

- **Vantagens:**

- **Eficiente** (usado em TLS 1.3, IPsec).
- **Autenticação integrada** (deteta alterações no criptograma).

Padding

- **Problema:** Mensagens com tamanho não múltiplo do bloco.
- **Soluções:**
 - **PKCS#5/PKCS#7:**
 - Adiciona X bytes com valor X (onde X = número de bytes necessários para completar o bloco).
 - Exemplo:
 - Bloco de 8 bytes, mensagem de 10 bytes → **6 bytes de padding** (valores $0x06$).
 - Se a mensagem for múltipla do bloco → adiciona **um bloco completo** de padding (ex: 8 bytes $0x08$).
 - **Ataques:**
 - **Padding Oracle Attack** (Vaudenay, 2002):
 - Explora erros na validação do *padding* para recuperar o texto plano.
 - **Contramedida:** Usar modos autenticados (ex: GCM).

2. Cifras Assimétricas e Assinaturas Digitais

2.1 Cifras Assimétricas

RSA (Rivest-Shamir-Adleman)

- **Base Matemática:**
 - **Fatorização de primos:** Dado $N = p * q$ (onde p e q são primos grandes), é **difícil** calcular p e q a partir de N .
 - **Teorema de Euler:** $a^{\phi(N)} \equiv 1 \pmod{N}$, onde $\phi(N) = (p-1)(q-1)$.
- **Geração de Chaves:**
 1. Escolher dois primos grandes p e q .
 2. Calcular $N = p * q$ e $\phi(N) = (p-1)(q-1)$.
 3. Escolher e (expoente público) tal que $1 < e < \phi(N)$ e $\gcd(e, \phi(N)) = 1$.
 4. Calcular d (expoente privado) tal que $d * e \equiv 1 \pmod{\phi(N)}$.
 5. **Chave pública:** (N, e) .
 6. **Chave privada:** (N, d) .
- **Cifra/Decifra:**
 - **Cifrar:** $c = m^e \pmod{N}$ (onde $0 \leq m < N$).
 - **Decifrar:** $m = c^d \pmod{N}$.
- **Exemplo Numérico:**
 - $p = 61, q = 53 \rightarrow N = 3233, \phi(N) = 3120$.
 - $e = 17 \rightarrow d = 2753$ (pois $17 * 2753 \equiv 1 \pmod{3120}$).
 - Cifrar $m = 65$: $c = 65^{17} \pmod{3233} = 2790$.
 - Decifrar $c = 2790$: $m = 2790^{2753} \pmod{3233} = 65$.
- **Segurança do RSA:**
 - **Ataques:**
 1. **Fatorização de N :**
 - Se o atacante conseguir fatorizar N , pode calcular $\phi(N)$ e d .
 - **Tamanho recomendado:** $N \geq 2048$ bits (atualmente, 4096 bits para alta segurança).
 2. **Ataques de Side-Channel:**
 - Medir tempo/consumo de energia para inferir d .
 - **Contramedida:** *Blinding* (multiplicar a mensagem por um valor aleatório antes de cifrar).

3. Ataques de *Chosen Ciphertext*:

- Atacante envia criptogramas para o decifrador e observa os resultados.
- Contramedida:** Usar **OAEP** (*Optimal Asymmetric Encryption Padding*).

- **Padding no RSA:**

- **PKCS#1 v1.5:**
 - Adiciona *padding* do tipo `0x00 0x01 [FF...FF] 0x00 [hash]`.
 - Vulnerável** a ataques de *chosen ciphertext* (ex: *Bleichenbacher Attack*).
- **OAEP (Optimal Asymmetric Encryption Padding):**
 - Usa *salt* aleatório e *hash* para preencher.
 - Seguro** contra ataques de *chosen ciphertext*.

- **Exemplo (OpenSSL):**

```
# Gerar um par de chaves RSA (2048 bits)
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048

# Extrair a chave pública
openssl rsa -pubout -in private_key.pem -out public_key.pem

# Cifrar uma mensagem com RSA (OAEP)
echo "Mensagem secreta" | openssl pkeyutl -encrypt -pubin -inkey public_key.pem -
pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256 -out ciphertext.bin

# Decifrar
openssl pkeyutl -decrypt -inkey private_key.pem -in ciphertext.bin -pkeyopt
rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

Diffie-Hellman (DH)

- **Objetivo:** Trocar uma **chave secreta** sem transmiti-la.
- **Funcionamento:**
 - Parâmetros públicos:**
 - p : Número primo grande (ex: 2048 bits).
 - g : Gerador (raiz primitiva módulo p).
 - Chaves privadas:**
 - Alice escolhe a (aleatório).
 - Bob escolhe b (aleatório).
 - Troca de chaves públicas:**
 - Alice envia $A = g^a \bmod p$.
 - Bob envia $B = g^b \bmod p$.
 - Cálculo do segredo compartilhado:**
 - Alice calcula $s = B^a \bmod p$.
 - Bob calcula $s = A^b \bmod p$.
 - Resultado:** $s = g^{(ab)} \bmod p$ (igual para ambos).
- **Segurança:**
 - Baseia-se na **difículdade do logaritmo discreto** (encontrar a dado $A = g^a \bmod p$).
 - Ataques:**
 - Ataque de *man-in-the-middle*:** Se não houver autenticação, um atacante pode trocar A e B .
 - Contramedida:** Usar **assinaturas digitais** (ex: no TLS, o servidor assina B).
- **Exemplo Numérico:**
 - $p = 23, g = 5$.

- Alice: $a = 6 \rightarrow A = 5^6 \bmod 23 = 8$.
- Bob: $b = 15 \rightarrow B = 5^{15} \bmod 23 = 19$.
- Segredo compartilhado:
 - Alice: $s = 19^6 \bmod 23 = 2$.
 - Bob: $s = 8^{15} \bmod 23 = 2$.

2.2 Assinaturas Digitais

Visão Geral

- **Objetivo:** Garantir **integridade + autenticidade + não-repúdio**.
- **Processo:**
 1. **Assinatura:**
 - Calcula-se o *hash* da mensagem: $h = H(m)$.
 - Cifra-se o *hash* com a **chave privada**: $s = E(k_{\text{priv}}, h)$.
 - **Assinatura final:** s (ou $s \parallel m$).
 2. **Verificação:**
 - Decifra-se s com a **chave pública**: $h' = D(k_{\text{pub}}, s)$.
 - Calcula-se o *hash* da mensagem recebida: $h'' = H(m')$.
 - ****Verifica-se se $h' == h''$ **.**
 - Se sim $\rightarrow$ **assinatura válida**.
 - Se não $\rightarrow$ **assinatura inválida** (mensagem alterada ou assinante não autorizado).
- **Fórmula:**

Assinar: $s = E(k_{\text{priv}}, H(m))$
 Verificar: $H(m') == D(k_{\text{pub}}, s)$

Propriedades de Segurança

1. Falsificação Existencial:

- Encontrar **qualquer** (m, s) tal que $V(k_{\text{pub}}, s, m) = \text{true}$.
- **Difícil** se a chave privada for segura.

2. Falsificação Seletiva:

- Dada uma mensagem m , encontrar s tal que $V(k_{\text{pub}}, s, m) = \text{true}$.
- **Difícil** se o *hash* for resistente a colisões.

Formatação (PKCS#1 v1.5 e PSS)

- **Problema:** O *hash* (ex: 256 bits) é menor que o módulo N do RSA (ex: 2048 bits).
- **Solução:** Usar **formatação** para preencher o *hash* até o tamanho de N .
 - **PKCS#1 v1.5:**
 - Adiciona *padding* do tipo $0x00\ 0x01\ [FF\dots FF]\ 0x00\ [hash]$.
 - **Vulnerável** a ataques de *chosen ciphertext*.
 - **PSS (Probabilistic Signature Scheme):**
 - Usa *salt* aleatório e *hash* para preencher.

- **Seguro** contra ataques de *chosen ciphertext*.

Exemplo (OpenSSL)

```
# Gerar um par de chaves RSA (para assinaturas)
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048

# Extrair a chave pública
openssl rsa -pubout -in private_key.pem -out public_key.pem

# Assinar um ficheiro
openssl dgst -sha256 -sign private_key.pem -out signature.bin ficheiro.txt

# Verificar a assinatura
openssl dgst -sha256 -verify public_key.pem -signature signature.bin ficheiro.txt

# Assinar com PSS
openssl dgst -sha256 -sign private_key.pem -pkeyopt rsa_padding_mode:pss -pkeyopt
rsa_pss_saltlen:20 -out signature.bin ficheiro.txt
```

JCA: Assinaturas Digitais (RSA)

```
import java.security.*;
import java.security.spec.*;

public class RSASignatureExample {
    public static void main(String[] args) throws Exception {
        // Gerar par de chaves RSA
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
        keyGen.initialize(2048);
        KeyPair pair = keyGen.generateKeyPair();
        PrivateKey privKey = pair.getPrivate();
        PublicKey pubKey = pair.getPublic();

        // Assinar uma mensagem
        Signature sig = Signature.getInstance("SHA256withRSA");
        sig.initSign(privKey);
        sig.update("Mensagem a assinar".getBytes());
        byte[] signature = sig.sign();

        // Verificar a assinatura
        sig.initVerify(pubKey);
        sig.update("Mensagem a assinar".getBytes());
        boolean isValid = sig.verify(signature);
        System.out.println("Assinatura válida? " + isValid);
    }
}
```


3.1 Funções Hash Criptográficas

Definição

- Função determinística que mapeia uma entrada de **tamanho arbitrário** para uma saída de **tamanho fixo**.
- Fórmula:** $H: \{0,1\}^* \rightarrow \{0,1\}^n$ (ex: SHA-256 $\rightarrow$ 256 bits).

Propriedades de Segurança

Propriedade	Definição	Importância
Resistência a Pré-imagem	Dado y , é difícil encontrar x tal que $H(x) = y$.	Evita ataques de <i>brute force</i> .
Resistência a Segunda Pré-imagem	Dado x , é difícil encontrar $x' \neq x$ tal que $H(x) = H(x')$.	Evita ataques de <i>dicionário</i> .
Resistência a Colisão	É difícil encontrar $x \neq x'$ tal que $H(x) = H(x')$.	Evita ataques de <i>birthday</i> .

Funções Hash Populares

Função	Tamanho da Saída	Estado Atual	Notas
MD5	128 bits	Obsoleta	Vulnerável a colisões (2004).
SHA-1	160 bits	Obsoleta	Vulnerável a colisões (2017).
SHA-256	256 bits	Segura	Usada em Bitcoin, TLS, SSL.
SHA-3	Variável	Segura	Baseada em <i>sponge functions</i> (Keccak).
BLAKE2	Variável	Segura	Mais rápida que SHA-3.
BLAKE3	Variável	Segura	Paralelizável, usada em <i>blockchain</i> .

Aplicações

1. Integridade de Dados:

- Verificar se um ficheiro foi alterado (ex: *downloads* de software).
- Exemplo:

```
echo -n "Mensagem" | openssl dgst -sha256
```

2. Assinaturas Digitais:

- Assinar o *hash* da mensagem (em vez da mensagem completa).

3. Armazenamento de Passwords:

- Armazenar $H(\text{password} + \text{salt})$ em vez da *password* em texto plano.

4. Estruturas de Dados:

- Tabelas *hash* (para busca eficiente).

5. Blockchain:

- Bitcoin usa SHA-256 para *Proof of Work*.

6. Prova de Trabalho (PoW):

- Mineração de Bitcoin: encontrar x tal que $H(x) < \text{alvo}$.

Ataques a Funções Hash

Ataque	Descrição	Exemplo
Colisão	Encontrar $x \neq x'$ com $H(x) = H(x')$.	SHA-1: colisões encontradas em 2017 (SHAttered Attack).
Pré-imagem	Dado y , encontrar x com $H(x) = y$.	Difícil para SHA-256.
Extensão de Hash	Dado $H(m)$, calcular $H(m$	
Rainbow Tables	Tabelas pré-computadas de <i>hashes</i> para ataques de dicionário.	Solução: usar <i>salt</i> .

Exemplo (OpenSSL)

```
# Calcular SHA-256 de um ficheiro
openssl dgst -sha256 ficheiro.txt

# Calcular SHA-256 de uma string
echo -n "Mensagem" | openssl dgst -sha256

# Verificar integridade de um ficheiro
echo "a1b2c3..." > expected_hash.txt
openssl dgst -sha256 -verify <(echo "a1b2c3...") ficheiro.txt
```

3.2 HMAC (Hash-based Message Authentication Code)

Definição

- **MAC**: *Message Authentication Code* (garante **integridade + autenticidade**).
- **HMAC**: MAC baseado em **função hash + chave simétrica**.
- **Fórmula**:

$$\text{HMAC}(k, m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$$

o

Calcular SHA-256 de um ficheiro

```
openssl dgst -sha256 ficheiro.txt
```

Calcular SHA-256 de uma string

```
echo -n "Mensagem" | openssl dgst -sha256
```

Verificar integridade de um ficheiro

```
echo "a1b2c3..." > expected_hash.txt openssl dgst -sha256 -verify <(echo "a1b2c3...") ficheiro.txt
```

3.2 HMAC (Hash-based Message Authentication Code)

Definição

- **MAC**: *Message Authentication Code* (garante **integridade + autenticidade**).
- **HMAC**: MAC baseado em **função hash** + chave simétrica.
- **Fórmula**:

$$\text{HMAC}(k, m) = H(k \oplus \text{opad} \parallel H(k \oplus \text{ipad} \parallel m))$$

- **opad** = `\0x5c5c5c...5c` (XOR com a chave).
- **ipad** = `\0x363636...36` (XOR com a chave).
- **H** = função **hash** (ex: SHA-256).

Propriedades

- Segurança**:
 - Se **H** for uma função **hash** segura (ex: SHA-256), o HMAC é **resistente a forjas**.
 - **Teorema de Segurança**: Se **H** for resistente a colisões, o HMAC é seguro (mesmo que **H** não seja resistente a pré-imagens).
- Tamanho da Saída**:
 - Igual ao tamanho da saída de **H** (ex: 256 bits para HMAC-SHA256).
- Eficiência**:
 - Mais rápido que assinaturas digitais (ex: RSA).

**Aplicações

- Integridade + Autenticidade de Mensagens**:
 - Usado em **TLS**, **IPsec**, **APIs REST** (ex: autenticação com **tokens** JWT).
- Autenticação de Cookies**:
 - Evitar que um atacante modifique **cookies** (ex: sessões **web**).
- Verificação de Chaves**:

- Confirmar que uma chave foi gerada corretamente.

Comparação: HMAC vs. Assinaturas Digitais

Característica	HMAC	Assinaturas Digitais (ex: RSA)
-----	-----	-----
Tipo	Simétrico	Assimétrico
Chave	1 chave simétrica	Par de chaves (pública/privada)
Velocidade	**Rápido**	Lento
Tamanho da Saída	Fixado pela função *hash*	Fixado pelo algoritmo (ex: 2048 bits)
Uso Típico	Autenticação de mensagens	Assinatura de documentos

Exemplo (OpenSSL)

```
```bash
Gerar HMAC-SHA256 de um ficheiro
openssl dgst -sha256 -hmac "minha_chave_secreta" ficheiro.txt

Gerar HMAC-SHA256 de uma string
echo -n "Mensagem" | openssl dgst -sha256 -hmac "minha_chave_secreta"

Verificar HMAC
echo -n "Mensagem" | openssl dgst -sha256 -hmac "minha_chave_secreta" -verify <(echo "hmac_esperado")
```

## JCA: HMAC

```
import javax.crypto.*;
import javax.crypto.spec.*;

public class MacExample {
 public static void main(String[] args) throws Exception {
 // Gerar uma chave para HMAC-SHA256
 KeyGenerator keyGen = KeyGenerator.getInstance("HmacSHA256");
 keyGen.init(256); // Tamanho da chave (em bits)
 SecretKey key = keyGen.generateKey();

 // Inicializar o MAC
 Mac mac = Mac.getInstance("HmacSHA256");
 mac.init(key);

 // Calcular o MAC
```

```
byte[] message = "Mensagem importante".getBytes();
byte[] macBytes = mac.doFinal(message);

System.out.println("MAC: " + bytesToHex(macBytes));
}

private static String bytesToHex(byte[] bytes) {
 StringBuilder sb = new StringBuilder();
 for (byte b : bytes) {
 sb.append(String.format("%02x", b));
 }
 return sb.toString();
}
}
```

---

## 4. Infraestrutura de Chave Pública (PKI)

---

### 4.1 Conceitos Básicos

#### Definição

- **PKI:** *Public Key Infrastructure* → Sistema para **gerir chaves públicas e certificados**.
  - **Objetivo:** Resolver o problema de **distribuição segura de chaves públicas** (evitar ataques *MITM*).
- 

#### Componentes da PKI

Componente	Descrição	Exemplo
<b>Resource Owner</b>	Entidade que <b>possui os recursos</b> (ex: Alice).	Utilizador final.
<b>Resource Server</b>	Servidor que <b>armazen os recursos</b> (ex: ``bash	

---

## Gerar HMAC-SHA256 de um ficheiro

```
openssl dgst -sha256 -hmac "minha_chave_secreta" ficheiro.txt
```

---

## Gerar HMAC-SHA256 de uma string

```
echo -n "Mensagem" | openssl dgst -sha256 -hmac "minha_chave_secreta"
```

---

## Verificar HMAC

```
echo -n "Mensagem" | openssl dgst -sha256 -hmac "minha_chave_secreta" -verify <(echo "hmac_esperado")
```

```

```

```
JCA: HMAC

```java
import javax.crypto.*;
import javax.crypto.spec.*;

public class MacExample {
    public static void main(String[] args) throws Exception {
        // Gerar uma chave para HMAC-SHA256
        KeyGenerator keyGen = KeyGenerator.getInstance("HmacSHA256");
        keyGen.init(256); // Tamanho da chave (em bits)
        SecretKey key = keyGen.generateKey();

        // Inicializar o MAC
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(key);

        // Calcular o MAC
        byte[] message = "Mensagem importante".getBytes();
        byte[] macBytes = mac.doFinal(message);

        System.out.println("MAC: " + bytesToHex(macBytes));
    }

    private static String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
```

4. Infraestrutura de Chave Pública (PKI)

4.1 Conceitos Básicos

Definição

- **PKI:** *Public Key Infrastructure* → Sistema para **gerir chaves públicas e certificados**.
- **Objetivo:** Resolver o problema de **distribuição segura de chaves públicas** (evitar ataques *MITM*).

Componentes da PKI

Componente	Descrição	Exemplo
Resource Owner	Entidade que possui os recursos (ex: Alice).	Utilizador final.
Resource Server	Servidor que armazen os recursos (ex: myphotos.com).	API REST.

Componente	Descrição	Exemplo
Authorization Server	Servidor que emite tokens (ex: <code>auth.myphotos.com</code>).	Mesmo que o <i>Resource Server</i> em muitos casos.
CA (Certification Authority)	Entidade que emite e assina certificados .	DigiCert, Let's Encrypt.
RA (Registration Authority)	Entidade que verifica a identidade antes de emitir um certificado.	Trabalha com a CA.
Certificado Digital	Documento que associa uma chave pública a uma identidade .	Formato X.509 .
CRL (Certificate Revocation List)	Lista de certificados revogados .	Verificada antes de aceitar um certificado.
OCSP (Online Certificate Status Protocol)	Protocolo para verificar o estado de um certificado em tempo real.	Alternativa à CRL.
Trust Anchor	Certificado raiz implicitamente confiável (ex: pré-instalado no <i>browser</i>).	Ex: Certificados da Mozilla, Microsoft.

4.2 Certificados X.509

Estrutura (ASN.1)

```

Certificate ::= SEQUENCE {
    tbsCertificate      TBSCertificate,
    signatureAlgorithm  AlgorithmIdentifier,
    signatureValue      BIT STRING
}

TBSCertificate ::= SEQUENCE {
    version             [0] EXPLICIT Version DEFAULT v1,
    serialNumber        CertificateSerialNumber,
    signature            AlgorithmIdentifier,
    issuer              Name,
    validity            Validity,
    subject             Name,
    subjectPublicKeyInfo SubjectPublicKeyInfo,
    issuerUniqueID      [1] IMPLICIT UniqueIdentifier OPTIONAL,
    subjectUniqueID     [2] IMPLICIT UniqueIdentifier OPTIONAL,
    extensions          [3] EXPLICIT Extensions OPTIONAL
}

```

Campos Importantes

Campo	Descrição	Exemplo
Version	Versão do certificado (v1, v2, v3).	``asn1

```

Certificate ::= SEQUENCE {

```

tbsCertificate

signatureAlgorithm

signatureValue

TBSCertificate,
AlgorithmIdentifier,
BIT STRING

}

TBSCertificate ::= SEQUENCE { version [0] EXPLICIT Version DEFAULT v1, serialNumber CertificateSerialNumber, signature AlgorithmIdentifier, issuer Name, validity Validity, subject Name, subjectPublicKeyInfo SubjectPublicKeyInfo, issuerUniqueID [1] IMPLICIT UniqueIdentifier OPTIONAL, subjectUniqueID [2] IMPLICIT UniqueIdentifier OPTIONAL, extensions [3] EXPLICIT Extensions OPTIONAL }

Campos Importantes

Campo	Descrição	Exemplo
-----	-----	-----
Version	Versão do certificado (v1, v2, v3).	`v3`
Serial Number	Número único do certificado (emitido pela CA).	`1234567890`
Issuer	Identidade da CA que emitiu o certificado.	`CN=Let's Encrypt, O=ISRG, C=US`
Subject	Identidade do proprietário do certificado.	`CN=example.com`
Validity	Período de validade (<code>`Not Before`</code> e <code>`Not After`</code>).	`2023-01-01` a `2024-01-01`
Public Key	Chave pública do proprietário (formato ASN.1).	`RSA Public Key`
Extensions	Campos adicionais (ex: <code>*Key Usage*</code> , <code>*SAN*</code>).	`Key Usage: Digital Signature`
Signature	Assinatura digital da CA sobre o <code>tbsCertificate`</code> .	`SHA256withRSA`

**Extensões Comuns

Extensão	Descrição
Exemplo	
-----	-----
Key Usage	Define para que o certificado pode ser usado (ex: assinatura, cifra). `Digital Signature, Key Encipherment`
Extended Key Usage	Usos estendidos (ex: autenticação de servidor, autenticação de cliente). `TLS Web Server Authentication`
Subject Alternative Name (SAN)	Nomes alternativos para o <code>*subject*</code> (ex: <code>`DNS:example.com`</code> , <code>`IP:192.168.1.1`</code>). `DNS:example.com, DNS:www.example.com`


```
| **Basic Constraints** | Indica se o *subject* é uma CA (`CA:TRUE`).  
| `CA:FALSE` |  
| **CRL Distribution Points** | URLs para obter a **CRL** da CA.  
| `http://crl.example.com/crl.pem` |  
  
---  
  
### **4.3 Cadeia de Certificação**  
  
#### **Processo de Validação**  
  
1. Receber um **certificado folha** (ex: `example.com`).  
2. Verificar a **assinatura** do certificado usando a **chave pública da CA emissora**.  
3. Obter o **certificado da CA emissora** (ex: `Let's Encrypt`).  
4. Repetir o processo até chegar a um **certificado raiz** (*trust anchor*).  
5. Se todas as assinaturas forem válidas e o *trust anchor* for confiável → **certificado válido**.  
  
---  
  
#### **Exemplo de Cadeia**
```

example.com (folha) ↓ (assinado por) Let's Encrypt R3 (intermédio) ↓ (assinado por) ISRG Root X1 (raiz, trust anchor)

```
---  
  
### **4.4 Formatos de Certificados**  
  
| Formato | Descrição | Extensão  
| Codificação |  
| ----- | ----- | -----  
| ----- | ----- |  
| **PEM** | Formato texto (Base64). |  
example.com (folha) |  
↓ (assinado por) |  
Let's Encrypt R3 (intermédio) |  
↓ (assinado por) |  
ISRG Root X1 (raiz, trust anchor) |
```

4.4 Formatos de Certificados

Formato	Descrição	Extensão	Codificação
PEM	Formato texto (Base64).	.pem, .crt, .cer	Base64
DER	Formato binário.	.der	Binário
PFX/PKCS#12	Container com chave privada + certificado + cadeia .	.pfx, .p12	Binário (cifrado)

Exemplo de Certificado PEM

```
-----BEGIN CERTIFICATE-----
MIIFazCCBF0gAwIBAgISA... (Base64)
-----END CERTIFICATE-----
```

4.5 Ataques e Contramedidas

Ataque	Descrição	Contramedida
MITM	Atacante substitui a chave pública de Alice pela sua própria.	Usar certificados digitais + cadeia de confiança .
Certificado Forjado	Atacante cria um certificado falso para ```.pem	

```
-----BEGIN CERTIFICATE-----
MIIFazCCBF0gAwIBAgISA...
(Base64)
-----END CERTIFICATE-----
```

```
---

### **4.5 Ataques e Contramedidas**

| Ataque          | Descrição |
| Contramedida    |           | |
|---|---|---|
| | ----- | |
| **MITM**          | Atacante substitui a chave pública de Alice pela sua
própria. | Usar **certificados digitais** + **cadeia de confiança**. |
| **Certificado Forjado** | Atacante cria um certificado falso para `example.com`.
Verificar a **assinatura da CA**. |
| **CA Comprometida**   | Atacante compromete uma CA e emite certificados falsos.
**CRL** e **OCSP** para revogar certificados. |
| **Certificado Expirado** | Certificado fora do período de validade.
Verificar o campo **Validity**. |
| **Certificado Revogado** | Certificado na CRL.
Verificar a **CRL** ou usar **OCSP**. |

---

### **4.6 Exemplo Prático (OpenSSL)**

```bash
Verificar um certificado
openssl x509 -in certificate.crt -text -noout

Verificar a cadeia de certificados de um servidor (HTTPS)
```

```
openssl s_client -connect example.com:443 -showcerts

Verificar a CRL de uma CA
openssl crl -in crl.pem -text -noout

Gerar um certificado auto-assinado (para testes)
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes -subj
"/CN=localhost"

Criar um CSR (Certificate Signing Request)
openssl req -new -newkey rsa:2048 -keyout key.pem -out csr.pem -subj "/CN=example.com"
```

---

## 4.7 JCA: Trabalhar com Certificados

### Ler um Certificado X.509

```
import java.io.*;
import java.security.cert.*;

public class CertificateExample {
 public static void main(String[] args) throws Exception {
 CertificateFactory cf = CertificateFactory.getInstance("X.509");
 try (FileInputStream fis = new FileInputStream("certificate.crt")) {
 X509Certificate cert = (X509Certificate) cf.generateCertificate(fis);
 System.out.println("Subject: " + cert.getSubjectX500Principal());
 System.out.println("Issuer: " + cert.getIssuerX500Principal());
 System.out.println("Valid from: " + cert.getNotBefore());
 System.out.println("Valid until: " + cert.getNotAfter());
 System.out.println("Public Key: " + cert.getPublicKey().getAlgorithm());
 }
 }
}
```

---

### Validar uma Cadeia de Certificados

```
import java.security.*;
import java.security.cert.*;
import java.util.*;

public class CertPathValidatorExample {
 public static void main(String[] args) throws Exception {
 CertificateFactory cf = CertificateFactory.getInstance("X.509");
 List<X509Certificate> certList = new ArrayList<>();
 certList.add(loadCert(cf, "leaf.crt")); // Certificado folha
 certList.add(loadCert(cf, "intermediate.crt")); // Certificado intermédio

 CertPath cp = cf.generateCertPath(certList);

 X509Certificate rootCert = loadCert(cf, "root.crt");
 Set<TrustAnchor> trustAnchors = new HashSet<>();
 trustAnchors.add(new TrustAnchor(rootCert, null));
 }
}
```

```

 PKIXParameters params = new PKIXParameters(trustAnchors);
 params.setRevocationEnabled(false); // Desabilitar CRL para exemplo

 CertPathValidator cpv = CertPathValidator.getInstance("PKIX");
 try {
 PKIXCertPathValidatorResult result = (PKIXCertPathValidatorResult)
cpv.validate(cp, params);
 System.out.println("✅ Cadeia válida!");
 } catch (CertPathValidatorException e) {
 System.out.println("❌ Cadeia inválida: " + e.getMessage());
 }
 }

 private static X509Certificate loadCert(CertificateFactory cf, String filename)
throws Exception {
 try (FileInputStream fis = new FileInputStream(filename)) {
 return (X509Certificate) cf.generateCertificate(fis);
 }
 }
}

```

## KeyStore para Certificados

```

import java.io.*;
import java.security.*;
import java.security.cert.*;

public class KeyStoreCertExample {
 public static void main(String[] args) throws Exception {
 KeyStore ks = KeyStore.getInstance("PKCS12");
 char[] password = "changeit".toCharArray();
 try (FileInputStream fis = new FileInputStream("keystore.p12")) {
 ks.load(fis, password);
 }

 String alias = "mycert";
 if (ks.containsAlias(alias)) {
 Certificate cert = ks.getCertificate(alias);
 System.out.println("Certificado: " + cert);
 if (cert instanceof X509Certificate) {
 X509Certificate x509Cert = (X509Certificate) cert;
 System.out.println("Subject: " + x509Cert.getSubjectX500Principal());
 }
 }
 }
}

```

## 5. TLS/SSL e HTTPS

## 5.1 Conceitos Básicos

### Definição

- **TLS**: *Transport Layer Security* → Protocolo para **comunicação segura** sobre redes inseguras (ex: Internet).
  - **SSL**: *Secure Sockets Layer* → Predecessor do TLS (obsoleto).
  - **HTTPS**: HTTP sobre TLS (porta **443**).
- 

### Objetivos do TLS

1. **Confidencialidade**: Cifrar os dados trocados.
  2. **Integridade**: Garantir que os dados não são alterados.
  3. **Autenticidade**: Verificar a identidade do servidor (e opcionalmente do cliente).
  4. **Não-repúdio**: (Opcional) Garantir que o servidor não pode negar uma ação.
- 

## 5.2 Arquitetura do TLS

- **Dois Sub-protocolos**:
  1. **Handshake Protocol**:
    - Negocia **chaves de sessão** e **parâmetros de segurança**.
    - Autentica o servidor (e opcionalmente o cliente).
  2. **Record Protocol**:
    - **Cifra e autentica** os dados trocados.
    - Usa *Cipher Suites* (ex: ``bash

## Verificar um certificado

---

```
openssl x509 -in certificate.crt -text -noout
```

## Verificar a cadeia de certificados de um servidor (HTTPS)

---

```
openssl s_client -connect example.com:443 -showcerts
```

## Verificar a CRL de uma CA

---

```
openssl crl -in crl.pem -text -noout
```

## Gerar um certificado auto-assinado (para testes)

---

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes -subj "/CN=localhost"
```

## Criar um CSR (Certificate Signing Request)

---

```
openssl req -new -newkey rsa:2048 -keyout key.pem -out csr.pem -subj "/CN=example.com"
```

```

4.7 JCA: Trabalhar com Certificados

Ler um Certificado X.509

```java
import java.io.*;
import java.security.cert.*;

public class CertificateExample {
    public static void main(String[] args) throws Exception {
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        try (FileInputStream fis = new FileInputStream("certificate.crt")) {
            X509Certificate cert = (X509Certificate) cf.generateCertificate(fis);
            System.out.println("Subject: " + cert.getSubjectX500Principal());
            System.out.println("Issuer: " + cert.getIssuerX500Principal());
            System.out.println("Valid from: " + cert.getNotBefore());
            System.out.println("Valid until: " + cert.getNotAfter());
            System.out.println("Public Key: " + cert.getPublicKey().getAlgorithm());
        }
    }
}

```

Validar uma Cadeia de Certificados

```

import java.security.*;
import java.security.cert.*;
import java.util.*;

public class CertPathValidatorExample {
    public static void main(String[] args) throws Exception {
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        List<X509Certificate> certList = new ArrayList<>();
        certList.add(loadCert(cf, "leaf.crt")); // Certificado folha
        certList.add(loadCert(cf, "intermediate.crt")); // Certificado intermédio

        CertPath cp = cf.generateCertPath(certList);

        X509Certificate rootCert = loadCert(cf, "root.crt");
        Set<TrustAnchor> trustAnchors = new HashSet<>();
        trustAnchors.add(new TrustAnchor(rootCert, null));

        PKIXParameters params = new PKIXParameters(trustAnchors);
        params.setRevocationEnabled(false); // Desabilitar CRL para exemplo

        CertPathValidator cpv = CertPathValidator.getInstance("PKIX");
        try {
            PKIXCertPathValidatorResult result = (PKIXCertPathValidatorResult)
                cpv.validate(cp, params);
            System.out.println("✅ Cadeia válida!");
        } catch (CertPathValidatorException e) {

```

```
        System.out.println("❌ Cadeia inválida: " + e.getMessage());
    }
}

private static X509Certificate loadCert(CertificateFactory cf, String filename)
throws Exception {
    try (FileInputStream fis = new FileInputStream(filename)) {
        return (X509Certificate) cf.generateCertificate(fis);
    }
}
}
```

KeyStore para Certificados

```
import java.io.*;
import java.security.*;
import java.security.cert.*;

public class KeyStoreCertExample {
    public static void main(String[] args) throws Exception {
        KeyStore ks = KeyStore.getInstance("PKCS12");
        char[] password = "changeit".toCharArray();
        try (FileInputStream fis = new FileInputStream("keystore.p12")) {
            ks.load(fis, password);
        }

        String alias = "mycert";
        if (ks.containsAlias(alias)) {
            Certificate cert = ks.getCertificate(alias);
            System.out.println("Certificado: " + cert);
            if (cert instanceof X509Certificate) {
                X509Certificate x509Cert = (X509Certificate) cert;
                System.out.println("Subject: " + x509Cert.getSubjectX500Principal());
            }
        }
    }
}
```

5. TLS/SSL e HTTPS

5.1 Conceitos Básicos

Definição

- **TLS**: *Transport Layer Security* → Protocolo para **comunicação segura** sobre redes inseguras (ex: Internet).
 - **SSL**: *Secure Sockets Layer* → Predecessor do TLS (obsoleto).
 - **HTTPS**: HTTP sobre TLS (porta **443**).
-

Objetivos do TLS

1. **Confidencialidade:** Cifrar os dados trocados.
 2. **Integridade:** Garantir que os dados não são alterados.
 3. **Autenticidade:** Verificar a identidade do servidor (e opcionalmente do cliente).
 4. **Não-repúdio:** (Opcional) Garantir que o servidor não pode negar uma ação.
-

5.2 Arquitetura do TLS

- **Dois Sub-protocolos:**
 1. **Handshake Protocol:**
 - Negocia **chaves de sessão e parâmetros de segurança**.
 - Autentica o servidor (e opcionalmente o cliente).
 2. **Record Protocol:**
 - **Cifra e autentica** os dados trocados.
 - Usa *Cipher Suites* (ex: **TLS_AES_256_GCM_SHA384**).
-

5.3 TLS Handshake (Simplificado)

1. Client Hello:

- Cliente envia:
 - Versão do TLS (ex: TLS 1.3).
 - Lista de *Cipher Suites* suportadas.
 - *Client Random* (32 bytes aleatórios).

2. Server Hello:

- Servidor escolhe:
 - *Cipher Suite* (da lista do cliente).
 - *Server Random* (32 bytes aleatórios).
- Envia o **certificado do servidor** (contém a chave pública).

3. Key Exchange:

- **RSA:** Cliente gera um *pre-master secret*, cifra com a chave pública do servidor e envia.
- **Diffie-Hellman:** Cliente e servidor trocam parâmetros DH e calculam o *pre-master secret*.

4. Derivação de Chaves:

- Ambos calculam:
- Derivam chaves para:
 - Cifra (ex: AES-GCM).
 - MAC (ex: HMAC-SHA256).
 - IVs.

5. Finished:

- Ambos enviam um *Finished* (HMAC de todas as mensagens anteriores).
 - Se os *Finished* coincidirem → **handshake bem-sucedido**.
-

5.4 Cipher Suites

<i>Cipher Suite</i>	Cifra	Modo	Hash	Key Exchange	PFS?	Notas
TLS_AES_256_GCM_SHA384	AES-256	GCM	SHA-384	RSA/ECDHE	✓	Usado em TLS 1.3.
TLS_CHACHA20_POLY1305_SHA256	ChaCha20	Poly1305	SHA-256	ECDHE	✓	Alternativa ao AES (melhor em <i>mobile</i>).
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384	AES-256	GCM	SHA-384	ECDHE	✓	Usa RSA para autenticação.
TLS_RSA_WITH_AES_256_CBC_SHA	AES-256	CBC	SHA-1	RSA	✗	Vulnerável (sem PFS).

5.5 Perfect Forward Secrecy (PFS)

- **Problema:**
 - Se um atacante gravar todo o tráfego TLS e depois **comprometer a chave privada do servidor**, poderá decifrar todas as sessões passadas.
- **Solução:**
 - Usar **Diffie-Hellman (DH)** ou **ECDHE** para trocar chaves **efêmeras** (que não são armazenadas).
 - **Exemplo:**
 - No TLS 1.3, **ECDHE** é obrigatório.
 - No TLS 1.2, *Cipher Suites* com **ECDHE** ou **DHE** fornecem PFS.

5.6 Ataques ao TLS e Contramedidas

Ataque	Descrição	Contramedida
BEAST	Ataque ao CBC no TLS 1.0 (explora <i>padding oracle</i>).	Usar TLS 1.1+ ou <i>Cipher Suites</i> com GCM.
POODLE	Ataque ao SSL 3.0 (explora <i>padding</i> no CBC).	Desabilitar SSL 3.0 .
Heartbleed	Explora um <i>bug</i> no OpenSSL (leitura de memória não inicializada).	Atualizar o OpenSSL.
FREAK	Força o uso de RSA-512 (fraco) em vez de RSA-2048.	Desabilitar <i>Cipher Suites</i> com chaves fracas.
Logjam	Ataque ao Diffie-Hellman com primos pequenos (ex: 512 bits).	Usar primos $\geq$ 2048 bits .
Sweet32	Ataque ao 3DES (bloco de 64 bits → vulnerável a colisões).	Usar AES em vez de 3DES.

5.7 TLS 1.3: Novidades

- **Remoção de *Cipher Suites* inseguras:**
 - **RSA** para *key exchange* (sem PFS) foi removido.
 - **CBC** foi removido (apenas **GCM** e **ChaCha20**).

- **Handshake mais rápido:**
 - **1-RTT** (Round-Trip Time) para conexões repetidas.
 - **0-RTT** (para dados que podem ser reenviados).
- **Obrigatoriedade de PFS:**
 - Todas as *Cipher Suites* usam **ECDHE** ou **DH**.
- **Novo Handshake:**
 - **Client Hello** + **Server Hello** em uma única mensagem.
 - **Derivação de chaves** mais simples (usando **HKDF**).

5.8 Exemplo Prático (OpenSSL)

```
# Criar um servidor HTTPS simples
openssl s_server -cert server.crt -key server.key -www -accept 4433

# Conectar a um servidor HTTPS (para depuração)
openssl s_client -connect example.com:443 -showcerts

# Listar as Cipher Suites suportadas por um servidor
openssl s_client -connect example.com:443 -tls1_3 -cipher DEFAULT

# Forçar o uso de uma Cipher Suite específica
openssl s_server -cert server.crt -key server.key -www -cipher TLS_AES_256_GCM_SHA384

# Gerar um par de chaves DH (para TLS)
openssl genpkey -algorithm DH -out dhparams.pem -pkeyopt dh_paramgen_prime_len:2048
```

5.9 JCA: TLS com SSLSocket

```
import javax.net.ssl.*;
import java.io.*;

public class TlsClientExample {
    public static void main(String[] args) throws Exception {
        SSLContext sslContext = SSLContext.getInstance("TLS");
        sslContext.init(null, null, null); // Usa o KeyStore padrão do Java

        SSLSocketFactory sf = sslContext.getSocketFactory();
        try (SSLSocket socket = (SSLSocket) sf.createSocket("example.com", 443)) {
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            out.println("GET / HTTP/1.1");
            out.println("Host: example.com");
            out.println();

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println(line);
            }
        }
    }
}
```

```
}  
}
```

6. Autenticação e Autorização

6.1 Autenticação Baseada em *Passwords*

Conceitos Básicos

Conceito	Definição	Exemplo
Autenticação	Processo de verificar a identidade de um utilizador.	<i>Password + username.</i>
Identificação	Processo de declarar uma identidade (ex: <i>username</i>).	Não implica autenticação.
Fator de Autenticação	Método usado para autenticar.	<i>Password, token, biometria.</i>
MFA	<i>Multi-Factor Authentication</i> : Usar múltiplos fatores .	<i>Password + SMS.</i>

Sistema de Autenticação

- **Arquitetura Típica:**

```
Utilizador → [Interface de Autenticação] → [Serviço de Autenticação] → [Base de  
Dados]
```

- **Interface de Autenticação:** Formulário *web* (ex: *login + password*).
- **Serviço de Autenticação:** Valida as credenciais (ex: compara *hashes*).
- **Base de Dados:** Armazena **informação de validação** (ex: *hash da password + salt*).

- **Funções:**

-

Criar um servidor HTTPS simples

```
openssl s_server -cert server.crt -key server.key -www -accept 4433
```

Conectar a um servidor HTTPS (para depuração)

```
openssl s_client -connect example.com:443 -showcerts
```

Listar as Cipher Suites suportadas por um servidor

```
openssl s_client -connect example.com:443 -tls1_3 -cipher DEFAULT
```

Forçar o uso de uma Cipher Suite específica

```
openssl s_server -cert server.crt -key server.key -www -cipher TLS_AES_256_GCM_SHA384
```

Gerar um par de chaves DH (para TLS)

```
openssl genpkey -algorithm DH -out dhparams.pem -pkeyopt dh_paramgen_prime_len:2048
```

```
---

### **5.9 JCA: TLS com SSLSocket**

```java
import javax.net.ssl.*;
import java.io.*;

public class TlsClientExample {
 public static void main(String[] args) throws Exception {
 SSLContext sslContext = SSLContext.getInstance("TLS");
 sslContext.init(null, null, null); // Usa o KeyStore padrão do Java

 SSLSocketFactory sf = sslContext.getSocketFactory();
 try (SSLSocket socket = (SSLSocket) sf.createSocket("example.com", 443)) {
 PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
 out.println("GET / HTTP/1.1");
 out.println("Host: example.com");
 out.println();

 BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
 String line;
 while ((line = in.readLine()) != null) {
 System.out.println(line);
 }
 }
 }
}
```

## 6. Autenticação e Autorização

### 6.1 Autenticação Baseada em *Passwords*

#### Conceitos Básicos

Conceito	Definição	Exemplo
<b>Autenticação</b>	Processo de <b>verificar a identidade</b> de um utilizador.	<i>Password + username.</i>

Conceito	Definição	Exemplo
Identificação	Processo de <b>declarar uma identidade</b> (ex: <i>username</i> ).	Não implica autenticação.
Fator de Autenticação	Método usado para autenticar.	<i>Password, token, biometria.</i>
MFA	<i>Multi-Factor Authentication</i> : Usar <b>múltiplos fatores</b> .	<i>Password + SMS.</i>

Sistema de Autenticação

• Arquitetura Típica:

Utilizador → [Interface de Autenticação] → [Serviço de Autenticação] → [Base de Dados]

- **Interface de Autenticação**: Formulário *web* (ex: *login + password*).
- **Serviço de Autenticação**: Valida as credenciais (ex: compara *hashes*).
- **Base de Dados**: Armazena **informação de validação** (ex: *hash da password + salt*).
- **Funções**:
  - $f(a)$ : Função que transforma a *password* **a** em **informação de validação** (ex:  $H(salt || a)$ ).
  - $g(v, a)$ : Função que verifica se  $f(a) == v$ .

Ataques a Passwords

Ataque	Descrição	Ferramentas
Força Bruta	Tentar todas as combinações possíveis.	Hashcat, John the Ripper.
Dicionário	Tentar <i>passwords</i> comuns (ex: "123456", "qwerty").	John the Ripper, Hashcat.
Rainbow Tables	Tabelas pré-computadas de <i>hashes</i> para <i>passwords</i> comuns.	RainbowCrack.
Pré-computação	Atacante pré-computa <i>hashes</i> para um dicionário de <i>passwords</i> .	-
Phishing	Enganar o utilizador para revelar a <i>password</i> .	-

Contramedidas

Contramedida	Descrição	Exemplo
Salt	Adicionar um valor aleatório à <i>password</i> antes de calcular o <i>hash</i> .	$H(salt$
Funções Lentas	Usar funções de <i>hash</i> <b>lentas</b> (ex: <i>Bcrypt, PBKDF2</i> ).	$Bcrypt(password, salt, cost=12)$ .
Políticas de Password	Exigir <i>passwords</i> <b>longas e complexas</b> (ex: 12+ caracteres).	Ferramentas: <b>zxcvbn</b> .
Limite de Tentativas	Bloquear após <b>N tentativas falhadas</b> (ex: 5).	Evita ataques de força bruta.

Contramedida	Descrição	Exemplo
<b>Backoff</b>	Aumentar o <b>tempo de espera</b> após cada tentativa falhada.	Ex: 1s → 2s → 4s → 8s.
<b>CAPTCHA</b>	Testes para distinguir humanos de <i>bots</i> .	Evita automação de ataques.
<b>MFA</b>	Usar <b>múltiplos fatores</b> (ex: <i>password</i> + <i>SMS</i> ).	Reduz o risco de comprometimento.

## Bcrypt

- **Características:**
  - **Função de *hash* lenta:** Usa **custo computacional** (ex: 12 rondas).
  - **Salt integrado:** Gera um *salt* aleatório para cada *password*.
  - **Formato:**

\$2a\$12\$N9qo8uL0ickgx2ZMRZoMy... (60 caracteres)

- **\$2a\$:** Versão do algoritmo.
- **12:** Custo (número de rondas).
- **N9qo8uL0ickgx2ZMRZoMy:** Salt (22 caracteres).
- **...:** Hash (31 caracteres).

- **Exemplo (Python):**

```
import bcrypt

password = b"minha_password"
salt = bcrypt.gensalt(rounds=12) # Custo = 12
hashed = bcrypt.hashpw(password, salt)

if bcrypt.checkpw(password, hashed):
 print("Password correta!")
```

## PBKDF2

- **Características:**
  - **Deriva uma chave** a partir de uma *password*.
  - Usa **HMAC + salt + iterações**.
  - **Formato:**
    - **PBKDF2(H, password, salt, iterations, dkLen).**
    - **H:** Função *hash* (ex: SHA-256).
    - **iterations:** Número de iterações (ex: 100.000).
    - **dkLen:** Tamanho da chave derivada (ex: 32 bytes para AES-256).
- **Exemplo (Python):**

```
import hashlib
import binascii
```

```
password = b"minha_password"
salt = binascii.unhexlify(b'aaf2d3f4e5c6d78') # Salt aleatório (16 bytes)
iterations = 100000
dkLen = 32 # Tamanho da chave (256 bits)

key = hashlib.pbkdf2_hmac('sha256', password, salt, iterations, dkLen)
print(binascii.hexlify(key))
```

### Exemplo: /etc/shadow (Linux)

- **Formato:**

```
username:password_hash:last_changed:min:max:warn:inactive:expire:reserved
```

- **Exemplo:**

```
alice:yj9T$KFXzGy0RXYho3ReDdB0P7.$YbRfWjHHnQRIId3Nt6Q1xLyeJC1iwGDafsRuo0YrAu/:1966
7:0:99999:7:::
```

- **\$y\$**: Algoritmo (*yescrypt*).
- **j9T**: Parâmetros.
- **KFXzGy0RXYho3ReDdB0P7.**: Salt.
- **YbRfWjHHnQRIId3Nt6Q1xLyeJC1iwGDafsRuo0YrAu/**: Hash da password.

### Exemplo Prático (John the Ripper)

```
Ataque de dicionário a /etc/shadow
john --format=crypt /etc/shadow

Usar um dicionário personalizado
john --wordlist=my_dict.txt --format=crypt /etc/shadow

Mostrar passwords quebradas
john --show /etc/shadow
```

## 6.2 Autenticação em Contexto Web

### Autenticação HTTP Basic/Digest

Tipo	Descrição	Vantagens	Desvantagens
Basic	Envia <i>username</i> + <i>password</i> <b>codificados em Base64</b> no header <i>Authorization</i> .	Simples.	<b>Inseguro</b> (textos plano codificados).

Tipo	Descrição	Vantagens	Desvantagens
<b>Digest</b>	Envia um <i>hash</i> da <i>password</i> (com <i>nonce</i> ).	Mais seguro que Basic.	Vulnerável a <i>replay attacks</i> .

- **Exemplo (Node.js - Basic Auth):**

```
const express = require('express');
const app = express();

app.get('/protected', (req, res) => {
 const authHeader = req.headers.authorization;
 if (!authHeader || !authHeader.startsWith('Basic ')) {
 res.set('WWW-Authenticate', 'Basic realm="Restricted Area"');
 return res.status(401).send('Authentication required');
 }

 const base64Credentials = authHeader.split(' ')[1];
 const credentials = Buffer.from(base64Credentials, 'base64').toString('ascii');
 const [username, password] = credentials.split(':');

 if (username === 'admin' && password === '123456') {
 res.send('Access granted!');
 } else {
 res.status(401).send('Invalid credentials');
 }
});

app.listen(3000);
```

## Autenticadores (Tokens)

- **Funcionamento:**

1. Utilizador envia *username* + *password* (ex: via formulário *web*).
2. Servidor valida as credenciais e gera um **autenticador** (ex: *token* aleatório).
3. Servidor envia o autenticador ao cliente (ex: em um *cookie*).
4. Cliente envia o autenticador em **todas as requisições subsequentes**.
5. Servidor valida o autenticador e autoriza o acesso.

- **Vantagens sobre HTTP Basic:**

- **Não envia credenciais** em cada requisição.
- **Pode ser revogado** (ex: *logout*).
- **Pode ter tempo de expiração**.

## Tipos de Autenticadores

Tipo	Descrição	Vantagens	Desvantagens
<b>Identificador de Sessão</b>	ID aleatório armazenado no servidor (ex: <i>sessionid=abc123</i> ).	Fácil de revogar.	Requer armazenamento no servidor.



Tipo	Descrição	Vantagens	Desvantagens
MAC (HMAC)	Token assinado com uma chave secreta (ex: <code>HMAC-SHA256(userid + timestamp)</code> ).	Sem armazenamento no servidor.	Token pode ser grande.
JWT	JSON Web Token: Token assinado (ou cifrado) com JSON.	Standardizado, sem estado.	Vulnerável se a chave for comprometida.

## JWT (JSON Web Token)

- Estrutura:

```
xxxxx.yyyyy.zzzzz
```

- **Header** (Base64): { "alg": "HS256", "typ": "JWT" }.
- **Payload** (Base64): { "sub": "1234567890", "name": "Alice", "iat": 1516239022 }.
- **Signature**: `HMAC-SHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)`.

- Claims Padrão:

Claim	Descrição	Obrigatório?
<code>iss</code>	Emissor ( <i>Issuer</i> ).	✓
<code>sub</code>	Subject (identificador único do utilizador).	✓
<code>aud</code>	Audiência ( <i>Audience</i> , ex: <code>client_id</code> ).	✓
<code>exp</code>	Data de expiração ( <i>Expiration Time</i> ).	✓
<code>nbf</code>	<i>Not Before</i> (data antes da qual o token é inválido).	✗
<code>iat</code>	<i>Issued At</i> (data de emissão).	✗
<code>jti</code>	JWT ID (identificador único do token).	✗

- Exemplo (Node.js):

```
const jwt = require('jsonwebtoken');
const secret = 'my_secret_key';

// Gerar um JWT
const token = jwt.sign({ userId: 123, username: 'alice' }, secret, { expiresIn: '1h' });

// Verificar um JWT
jwt.verify(token, secret, (err, decoded) => {
 if (err) {
 console.log('Token inválido:', err);
 } else {
 console.log('Token válido:', decoded);
 }
});
```

```
}
});
```

## CSRF (Cross-Site Request Forgery)

- **Ataque:**

1. Utilizador está autenticado em `example.com` (com *cookie* de sessão).
2. Atacante convence o utilizador a visitar `evil.com`.
3. `evil.com` envia uma requisição para `example.com` (ex: ``).
4. O *browser* envia o *cookie* de sessão automaticamente → **requisição é autorizada**.

- **Contramedidas:**

1. **CSRF Token:**

- Servidor gera um *token* aleatório e armazena-o na sessão.
- Inclui o *token* em formulários (ex: `<input type="hidden" name="csrf_token" value="abc123">`).
- Valida o *token* em cada requisição.

2. **SameSite Cookies:**

- *Cookie* só é enviado se o *site* de origem for o mesmo.
- **Set-Cookie:** `sessionid=abc123; SameSite=Strict` (ou `Lax`).

3. **Double Submit Cookie:**

- Envia o *token* tanto no *cookie* quanto no *header* (ex: `X-CSRF-Token`).
- Valida que ambos são iguais.

- **Exemplo (CSRF Token em Node.js):**

```
const csrf = require('csrf');
const cookieParser = require('cookie-parser');

app.use(cookieParser());
app.use(csrf({ cookie: true }));

app.get('/form', (req, res) => {
 res.send(`
 <form action="/submit" method="POST">
 <input type="hidden" name="_csrf" value="${req.csrfToken()}">
 <input type="text" name="data">
 <button type="submit">Submit</button>
 </form>
 `);
});

app.post('/submit', (req, res) => {
 res.send('Data submitted!');
});
```

## Segurança de Cookies

Propriedade	Descrição	Exemplo
<b>HttpOnly</b>	<i>Cookie não é acessível</i> via JavaScript ( <code>document.cookie</code> ).	<code>Set-Cookie: sessionid=abc123; HttpOnly</code>
<b>Secure</b>	<i>Cookie só é enviado via HTTPS.</i>	<code>Set-Cookie: sessionid=abc123; Secure</code>
<b>SameSite</b>	<i>Cookie só é enviado se o site for o mesmo</i> ( <code>Strict</code> , <code>Lax</code> , <code>None</code> ).	<code>Set-Cookie: sessionid=abc123; SameSite=Strict</code>
<b>Expires/Max-Age</b>	Data de expiração do <i>cookie</i> .	<code>Set-Cookie: sessionid=abc123; Max-Age=3600</code> (1 hora)

### Exemplo: Autenticação com Cookies (Node.js + Express)

```
const express = require('express');
const session = require('express-session');
const app = express();

app.use(session({
 secret: 'my_secret_key',
 resave: false,
 saveUninitialized: false,
 cookie: { secure: true, httpOnly: true } // HTTPS + HttpOnly
}));

app.get('/login', (req, res) => {
 res.send(`
 <form action="/login" method="POST">
 <input type="text" name="username" placeholder="Username">
 <input type="password" name="password" placeholder="Password">
 <button type="submit">Login</button>
 </form>
 `);
});

app.post('/login', express.urlencoded({ extended: true }), (req, res) => {
 const { username, password } = req.body;
 if (username === 'admin' && password === '123456') {
 req.session.user = username;
 res.redirect('/protected');
 } else {
 res.status(401).send('Invalid credentials');
 }
});

app.get('/protected', (req, res) => {
 if (!req.session.user) {
 return res.status(401).send('Unauthorized');
 }
 res.send(`Welcome, ${req.session.user}! <a href="/logout">Logout</a>`);
});

app.get('/logout', (req, res) => {
 req.session.destroy();
 res.redirect('/login');
});
```

```
});

app.listen(3000);
```

## 6.3 OAuth 2.0

### Conceitos Básicos

Conceito	Descrição	Exemplo
<b>OAuth 2.0</b>	<i>Framework</i> para <b>autorização delegada</b> .	Usado por Google, Facebook, GitHub.
<b>Resource Owner</b>	Entidade que <b>possui os recursos</b> (ex: Alice).	Utilizador final.
<b>Client</b>	Aplicação que <b>requisita acesso</b> aos recursos (ex: <code>printonline.com</code> ).	App <i>web</i> ou <i>mobile</i> .
<b>Resource Server</b>	Servidor que <b>armazen os recursos</b> (ex: <code>myphotos.com</code> ).	API REST.
<b>Authorization Server</b>	Servidor que <b>emite tokens</b> (ex: <code>auth.myphotos.com</code> ).	Mesmo que o <i>Resource Server</i> em muitos casos.
<b>Access Token</b>	<i>Token</i> que <b>autoriza o client a aceder aos recursos</b> .	<b>Opaque</b> (o <i>client</i> não sabe o conteúdo).
<b>Refresh Token</b>	<i>Token</i> usado para <b>obter novos access tokens</b> (quando estes expiram).	<b>Longo prazo</b> (armazena-se com segurança).
<b>Scope</b>	<b>Permissões</b> solicitadas pelo <i>client</i> (ex: <code>read</code> , <code>write</code> ).	Ex: <code>scope=photos.readonly</code> .
<b>Client ID</b>	Identificador <b>público</b> do <i>client</i> (registado na <i>Authorization Server</i> ).	Ex: <code>client_id=abc123</code> .
<b>Client Secret</b>	<b>Segredo</b> do <i>client</i> (usado para autenticar-se junto à <i>Authorization Server</i> ).	<b>Confidencial</b> (não deve ser exposto).

### Fluxos de Autorização (Grant Types)

Fluxo	Descrição	Quando Usar	Segurança
<b>Authorization Code</b>	Usa um <b>código de autorização</b> para obter um <i>access token</i> .	Aplicações <i>web</i> (servidor).	★★★★★
<b>Implicit</b>	Retorna o <i>access token</i> <b>diretamente</b> no <i>fragment</i> da URL.	Aplicações <i>single-page</i> (SPA).	★★★ (evitar)
<b>Resource Owner Password Credentials</b>	Usa <b>username + password</b> do <i>resource owner</i> para obter um <i>token</i> .	Aplicações confiáveis (ex: <i>desktop</i> ).	★★ (evitar)
<b>Client Credentials</b>	Usa <b>client_id + client_secret</b> para obter um <i>token</i> (sem <i>resource owner</i> ).	Aplicações <i>server-to-server</i> .	★★★★

### Fluxo Authorization Code (Detalhado)

### 1. Client redireciona o *Resource Owner* para o *Authorization Server*:

- URL:
- **Parâmetros:**
  - `response_type=code`: Solicita um *authorization code*.
  - `client_id`: Identificador do *client*.
  - `redirect_uri`: URL para onde o *Authorization Server* deve redirecionar.
  - `scope`: Permissões solicitadas.
  - `state`: *Token* CSRF (para evitar ataques).

### 2. *Resource Owner* autentica-se e autoriza o *Client*:

- O *Authorization Server* mostra uma página de *login* e solicita permissão.
- O *Resource Owner* faz *login* e **aprova** as permissões.

### 3. *Authorization Server* redireciona de volta para o *Client*:

- URL:
- O *client* verifica que o `state` é o mesmo que foi enviado.

### 4. Client troca o *Authorization Code* por um *Access Token*:

- Requisição (POST):
- Resposta:

```
{
 "access_token": "ACCESS_TOKEN",
 "token_type": "Bearer",
 "expires_in": 3600,
 "refresh_token": "REFRESH_TOKEN"
}
```

### 5. Client usa o *Access Token* para aceder ao *Resource Server*:

- Requisição:

---

## Exemplo: OAuth 2.0 com Node.js (Authorization Code)

```
const express = require('express');
const axios = require('axios');
const app = express();
const clientId = 'YOUR_CLIENT_ID';
const clientSecret = 'YOUR_CLIENT_SECRET';
const redirectUri = 'http://localhost:3000/callback';

app.get('/login', (req, res) => {
 const state = Math.random().toString(36).substring(2);
 req.session.state = state;
 res.redirect(`https://github.com/login/oauth/authorize?
 client_id=${clientId}&
 redirect_uri=${encodeURIComponent(redirectUri)}&
 scope=repo&
 state=${state}`);
});
```

```
});

app.get('/callback', async (req, res) => {
 const { code, state } = req.query;
 if (state !== req.session.state) {
 return res.status(403).send('Invalid state');
 }

 const response = await axios.post('https://github.com/login/oauth/access_token',
 null, {
 params: {
 client_id: clientId,
 client_secret: clientSecret,
 code: code,
 redirect_uri: redirectUri
 }
 });

 const { access_token } = response.data;
 req.session.accessToken = access_token;
 res.redirect('/profile');
});

app.get('/profile', async (req, res) => {
 const { accessToken } = req.session;
 const response = await axios.get('https://api.github.com/user', {
 headers: {
 Authorization: `Bearer ${accessToken}`
 }
 });
 res.json(response.data);
});

app.listen(3000);
```

## Segurança no OAuth 2.0

Risco	Descrição	Contramedida
<b>Client Secret Exposto</b>	<i>Client secret</i> é incluído no código ou em repositórios públicos.	Armazenar em <b>variáveis de ambiente</b> ou <i>Key Vault</i> .
<b>Authorization Code Interceptado</b>	Atacante intercepta o <i>authorization code</i> no redirecionamento.	Usar <b>HTTPS</b> + <b>state</b> (CSRF token).
<b>Access Token Roubado</b>	Atacante rouba o <i>access token</i> (ex: via XSS ou logs).	Usar <b>tokens de curta duração</b> + <i>refresh tokens</i> .
<b>CSRF</b>	Atacante engana o utilizador a autorizar um <i>client</i> malicioso.	Usar <b>**state**</b> (token CSRF).
<b>Phishing</b>	Atacante cria um <i>Authorization Server</i> falso.	Educar utilizadores + <b>PKI</b> (certificados válidos).
<b>Token Replay</b>	Atacante reutiliza um <i>access token</i> válido.	Usar <b>tokens de uso único</b> ou <i>nonce</i> .

## PKCE (Proof Key for Code Exchange)

- **Objetivo:** Proteger *public clients* (ex: SPAs, apps *mobile*) que não podem armazenar um `client_secret` com segurança.
- **Funcionamento:**
  1. Client gera um `code_verifier` (aleatório, 43-128 caracteres).
  2. Client calcula o `code_challenge` = `SHA256(code_verifier)` (Base64 URL-safe).
  3. Client envia o `code_challenge` no *authorization request*.
  4. Authorization Server armazena o `code_challenge`.
  5. Client envia o `code_verifier` ao trocar o *authorization code* por um *token*.
  6. Authorization Server verifica que `SHA256(code_verifier) == code_challenge`.
- **Exemplo (Node.js):**

```
const crypto = require('crypto');

app.get('/login', (req, res) => {
 const codeVerifier = crypto.randomBytes(32).toString('base64url');
 const codeChallenge = crypto.createHash('sha256')
 .update(codeVerifier)
 .digest('base64url');

 req.session.codeVerifier = codeVerifier;

 const authUrl = `https://auth-server.com/authorize?
 response_type=code&
 client_id=CLIENT_ID&
 redirect_uri=${encodeURIComponent(redirectUri)}&
 code_challenge=${codeChallenge}&
 code_challenge_method=S256`;
 res.redirect(authUrl);
});

app.get('/callback', async (req, res) => {
 const { code } = req.query;
 const codeVerifier = req.session.codeVerifier;

 const response = await axios.post('https://auth-server.com/token', null, {
 params: {
 grant_type: 'authorization_code',
 code: code,
 redirect_uri: redirectUri,
 client_id: clientId,
 code_verifier: codeVerifier
 }
 });

 const { access_token } = response.data;
 res.json({ access_token });
});
```

---

## 6.4 OpenID Connect

### Conceitos Básicos

Conceito	Descrição	Exemplo
<b>OpenID Connect (OIDC)</b>	<b>Camada de identidade</b> sobre OAuth 2.0.	Usado por Google, Microsoft, Auth0.
<b>ID Token</b>	<i>Token</i> JWT que <b>confirma a identidade</b> do utilizador.	Contém <i>claims</i> como <b>sub</b> , <b>name</b> , <b>email</b> .
<b>UserInfo Endpoint</b>	API que retorna <b>informações adicionais</b> sobre o utilizador.	Acessado com o <i>access token</i> .
<b>Relying Party (RP)</b>	Aplicação que <b>confia</b> no <i>OpenID Provider</i> para autenticar utilizadores.	Ex: <b>myapp.com</b> .
<b>OpenID Provider (OP)</b>	Servidor que <b>autentica utilizadores</b> e emite <i>ID Tokens</i> .	Ex: Google, Auth0.
<b>Discovery Endpoint</b>	Endpoint que fornece a <b>configuração do OP</b> .	<b><i>/.well-known/openid-configuration</i></b> .
<b>JWKS Endpoint</b>	Endpoint que fornece as <b>chaves públicas</b> do OP.	Usado para validar assinaturas JWT.

### Diferenças: OAuth 2.0 vs. OpenID Connect

Característica	OAuth 2.0	OpenID Connect
<b>Objetivo</b>	Autorização (acesso a recursos).	<b>Autenticação</b> (identidade).
<b>Tokens</b>	<i>Access Token</i> (opaco).	<i>Access Token</i> + <b>ID Token</b> (JWT).
<b>Endpoints</b>	<b><i>/authorize, /token</i></b> .	<b><i>/authorize, /token, **/userinfo**</i></b> .
<b>Scopes</b>	<b><i>read, write</i></b> .	<b><i>openid, profile, email</i></b> .
<b>Uso Típico</b>	Aplicações que precisam de acesso a APIs.	Aplicações que precisam de <b>login</b> .

### Fluxo OpenID Connect (Authorization Code)

#### 1. RP redireciona o utilizador para o OP:

- URL:
- **Parâmetros:**
  - ***scope=openid***: **Obrigatório** para OpenID Connect.
  - ***nonce***: *Token* para evitar *replay attacks*.

#### 2. Utilizador autentica-se e autoriza o RP:

- O OP mostra uma página de *login* e solicita permissão.
- O utilizador faz *login* e **aprova** as permissões.

#### 3. OP redireciona de volta para o RP:

- URL:

#### 4. RP troca o *Authorization Code* por *ID Token* + *Access Token*:



- Requisição (POST):
- Resposta:

```
{
 "access_token": "ACCESS_TOKEN",
 "token_type": "Bearer",
 "expires_in": 3600,
 "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
 "refresh_token": "REFRESH_TOKEN"
}
```

#### 5. RP valida o ID Token:

- Decodificar o JWT (ex: usando [jwt.io](https://jwt.io)).
- Verificar:
  - **Assinatura** (usando a chave pública do OP).
  - **iss** (emissor) = URL do OP.
  - **aud** (audiência) = **client\_id** do RP.
  - **nonce** (se foi enviado no passo 1).
  - **exp** (expiração) > data atual.

#### 6. RP usa o Access Token para obter UserInfo:

- Requisição:
- Resposta:

```
{
 "sub": "1234567890",
 "name": "Alice",
 "email": "alice@example.com"
}
```

---

### Exemplo: OpenID Connect com Node.js

```
const express = require('express');
const { Issuer } = require('openid-client');
const app = express();

app.get('/login', async (req, res) => {
 const op = await Issuer.discover('https://op.example.com');
 const client = new op.Client({
 client_id: 'CLIENT_ID',
 client_secret: 'CLIENT_SECRET',
 redirect_uris: ['http://localhost:3000/callback']
 });

 const state = Math.random().toString(36).substring(2);
 const nonce = Math.random().toString(36).substring(2);
 req.session.state = state;
 req.session.nonce = nonce;
```

```
const authorizationUrl = client.authorizationUrl({
 scope: 'openid profile email',
 state: state,
 nonce: nonce
});
res.redirect(authorizationUrl);
});

app.get('/callback', async (req, res) => {
 const { code, state } = req.query;
 if (state !== req.session.state) {
 return res.status(403).send('Invalid state');
 }

 const op = await Issuer.discover('https://op.example.com');
 const client = new op.Client({
 client_id: 'CLIENT_ID',
 client_secret: 'CLIENT_SECRET',
 redirect_uris: ['http://localhost:3000/callback']
 });

 const params = client.callbackParams(req);
 const tokenSet = await client.callback('http://localhost:3000/callback', params, {
 nonce: req.session.nonce
 });

 const claims = tokenSet.claims();
 console.log('ID Token Claims:', claims);

 const userinfo = await client.userinfo(tokenSet);
 console.log('UserInfo:', userinfo);

 req.session.user = userinfo;
 res.redirect('/profile');
});

app.listen(3000);
```

## Segurança no OpenID Connect

Risco	Descrição	Contramedida
<b>ID Token Forjado</b>	Atacante cria um <i>ID Token</i> falso.	Validar <b>assinatura</b> + <b>**iss**</b> , <b>**aud**</b> , <b>**nonce**</b> .
<b>Replay Attack</b>	Atacante reutiliza um <i>ID Token</i> válido.	Usar <b>**nonce**</b> + <b>**iat**</b> (verificar tempo).
<b>Phishing</b>	Atacante cria um OP falso.	Educar utilizadores + <b>PKI</b> (certificados válidos).
<b>Token Theft</b>	Atacante rouba o <i>ID Token</i> ou <i>Access Token</i> .	Usar <b>tokens de curta duração</b> + <b>HTTPS</b> .

## Exemplo: Validar um ID Token (JWT) em Node.js

```
const jwt = require('jsonwebtoken');
const jwksClient = require('jwks-rsa');

const client = jwksClient({
 jwksUri: 'https://op.example.com/.well-known/jwks.json'
});

function getKey(header, callback) {
 client.getSigningKey(header.kid, (err, key) => {
 const signingKey = key.publicKey || key.rsaPublicKey;
 callback(null, signingKey);
 });
}

app.post('/validate-token', (req, res) => {
 const { idToken } = req.body;
 jwt.verify(idToken, getKey, {
 issuer: 'https://op.example.com',
 audience: 'CLIENT_ID'
 }, (err, decoded) => {
 if (err) {
 return res.status(401).send('Token inválido');
 }
 res.json(decoded);
 });
});
```

---

## 6.5 Ataques contra OAuth 2.0 e OpenID Connect

### Modelo de Atacante (RFC 6819)

- **Hipóteses:**
  1. **Acesso à rede:**
    - Atacante pode **monitorizar** e **modificar** tráfego entre *client* e *authorization server*.
    - **Não pode** aceder à comunicação entre *authorization server* e *resource server*.
  2. **Recursos ilimitados:**
    - Atacante pode configurar servidores maliciosos.
  3. **Conluio:**
    - Até **2 de 3 entidades** (*client*, *resource owner*, *authorization server*) podem estar sob controlo do atacante.

---

### Ataques Comuns

Ataque	Descrição	Contramedida
<b>CSRF</b>	Atacante engana o utilizador a autorizar um <i>client</i> malicioso.	Usar <b>**state**</b> (token CSRF).
<b>Token Theft</b>	Atacante rouba o <i>access token</i> ou <i>refresh token</i> .	Usar <b>tokens de curta duração</b> + <b>PKCE</b> .
<b>Client Secret Exposto</b>	<i>Client secret</i> é incluído no código ou em repositórios públicos.	Armazenar em <b>variáveis de ambiente</b> ou <i>Key Vault</i> .

Ataque	Descrição	Contramedida
<b>Phishing</b>	Atacante cria um <i>Authorization Server</i> falso.	Educar utilizadores + <b>PKI</b> (certificados válidos).
<b>Token Replay</b>	Atacante reutiliza um <i>access token</i> válido.	Usar <b>tokens de uso único</b> ou <i>nonce</i> .
<b>Open Redirector</b>	Atacante explora um <i>open redirector</i> no <i>Authorization Server</i> ou <i>Client</i> .	Validar <b>**redirect_uri**</b> .

## 7. Controlo de Acesso

### 7.1 Conceitos Básicos

#### Definição

- **Controlo de Acesso:** Mecanismos para **restringir e auditar** o acesso a recursos.
- **Componentes:**
  - **PEP** (*Policy Enforcement Point*): **Interceta** requisições e aplica políticas.
  - **PDP** (*Policy Decision Point*): **Decide** se uma requisição é permitida.

#### Arquitetura: Monitor de Referências

Sujeito → [PEP] → [PDP] → Objeto

- **PEP:**
  - Interceta a requisição.
  - Extrai informações (sujeito, objeto, ação).
  - Consulta o **PDP**.
  - **Permite ou nega** o acesso.
- **PDP:**
  - Recebe a requisição do **PEP**.
  - Consulta as **políticas de controlo de acesso**.
  - Retorna uma **decisão** (permitir/negar).

### 7.2 Modelos de Controlo de Acesso

#### 1. Modelo de Matriz de Acessos

- **Representação:**
  - Matriz onde:
    - **Linhas** = Sujeitos.
    - **Colunas** = Objetos.
    - **Células** = Permissões (ex: {**read**, **write**}).
- **Exemplo:**

Sujeito	File1	File2	Program1
Alice	R, W	R, W	X
Bob	R		
Charlie		R	X, W

- **Implementações:**

1. **Tabela de Autorização:**

- Lista de triplos (**Sujeito**, **Permissão**, **Objeto**).
- **Vantagem:** Fácil de implementar.
- **Desvantagem:** Pesquisa lenta ( $O(n)$ ).

2. **Capabilities:**

- Cada sujeito tem uma **lista de permissões** (linha da matriz).
- **Vantagem:** Pesquisa rápida para um sujeito ( $O(1)$ ).
- **Desvantagem:** Difícil listar permissões para um objeto.

3. **ACL (Access Control List):**

- Cada objeto tem uma **lista de permissões** (coluna da matriz).
- **Vantagem:** Pesquisa rápida para um objeto ( $O(1)$ ).
- **Desvantagem:** Difícil listar permissões para um sujeito.

## 2. Modelo RBAC (Role-Based Access Control)

- **Ideia:** Permissões são atribuídas a **papeis** (*roles*), e sujeitos herdam permissões dos seus papeis.
- **Vantagens:**
  - **Escalável:** Gerir permissões por papeis (ex: "Admin", "User") em vez de por utilizador.
  - **Flexível:** Utilizadores podem ter **múltiplos papeis**.
  - **Hierarquia de papeis:** Papeis podem herdar permissões de outros papeis (ex: "Admin" herda de "User").
- **Níveis do RBAC** (Sandhu et al., 1996):

Nível	Descrição	Características
<b>RBAC<sub>0</sub></b>	Modelo base.	Sujeitos, papeis, permissões.
<b>RBAC<sub>1</sub></b>	Adiciona <b>hierarquia de papeis</b> .	Papeis podem herdar permissões.
<b>RBAC<sub>2</sub></b>	Adiciona <b>restrições</b> .	Ex: "Separation of Duty" (SoD).
<b>RBAC<sub>3</sub></b>	Combina RBAC <sub>1</sub> + RBAC <sub>2</sub> .	Hierarquia + restrições.

- **Exemplo (RBAC<sub>1</sub>):**
  - **Papeis:**
    - **Admin** (herda de **User**).
    - **User**.
  - **Permissões:**
    - **User:** **read** em **File1**.
    - **Admin:** **write** em **File1**.
  - **Atribuição:**
    - Alice → **Admin**.

- Bob → **User**.
  - **Resultado:**
    - Alice: **read, write** em **File1**.
    - Bob: **read** em **File1**.
  - **Restrições (RBAC<sub>2</sub>):**
    - **Separation of Duty (SoD):**
      - Um utilizador **não pode ter** dois papéis que conflitem (ex: "Approver" e "Requester").
    - **Cardinality Constraints:**
      - Limitar o número de utilizadores em um papel (ex: máximo 2 utilizadores no papel "Admin").
    - **Prerequisite Roles:**
      - Um utilizador só pode ser atribuído a um papel se já tiver outro (ex: "Senior" requer "Junior").
- 

### 3. Modelo ABAC (Attribute-Based Access Control)

- **Ideia:** Permissões são baseadas em **atributos** (ex: **department=Finance, time=9-17**).
  - **Vantagens:**
    - **Flexível:** Políticas baseadas em **contexto** (ex: hora do dia, localização).
    - **Dinâmico:** Permissões podem mudar com base em atributos (ex: **clearance=TopSecret**).
  - **Exemplo:**
    - **Política:**
      - **Permitir read em File1 SE department=Finance AND time=9-17.**
    - **Atributos:**
      - Sujeito: **Alice** (department=Finance, clearance=Secret).
      - Objeto: **File1** (classification=Confidential).
      - Contexto: **time=10:00**.
    - **Resultado: Permitido** (department=Finance e time=9-17).
- 

### 7.3 Estudo de Caso: Controlo de Acesso no Linux

- **Ficheiros:**
  - **/etc/passwd:** Informações dos utilizadores (UID, GID, *home, shell*).
  - **/etc/shadow:** Hashes das *passwords* (apenas legível por *root*).
  - **/etc/group:** Grupos e seus membros.
- **Permissões de Ficheiros:**
  - **Owner** (dono): Permissões para o utilizador dono.
  - **Group:** Permissões para o grupo dono.
  - **Others:** Permissões para os restantes utilizadores.
- **Exemplo:**

```
$ ls -l
-rw-r--r-- 1 alice users 1024 Jan 1 10:00 file1.txt
```

- **alice** (dono): **rw-** (read + write).
  - **users** (grupo): **r--** (read).
  - **others:** **r--** (read).
-

### 7.4 Estudo de Caso: Controlo de Acesso no Windows

- **Componentes:**
  - **Access Token:** Associado a um utilizador (contém SID, grupos, privilégios).
  - **Security Descriptor:** Associado a um objeto (contém DACL, SACL, owner, group).
- **DACL (Discretionary Access Control List):**
  - Lista de **ACE** (*Access Control Entries*).
  - Cada ACE contém:
    - **SID** (utilizador/grupo).
    - **Permissões** (ex: **READ**, **WRITE**).
    - **Tipo** (Allow/Deny).
  - **Processamento:** O Windows **percorre as ACEs em ordem** e aplica a primeira que casar.
- **SACL (System Access Control List):**
  - Usado para **auditoria** (registar acessos).
- **Exemplo de DACL:**

```
ACE 1: Allow User1 READ
ACE 2: Deny GroupA WRITE
ACE 3: Allow Everyone READ
```

- Se **User1** pertence a **GroupA**:
  - Acesso **READ** → **Permitido** (ACE 1).
  - Acesso **WRITE** → **Negado** (ACE 2).

### 7.5 Comparação: RBAC vs. ABAC vs. ACL

Característica	RBAC	ABAC	ACL
Baseado em	Papeis	Atributos	Listas de acesso
Flexibilidade	Média	Alta	Baixa
Escalabilidade	Alta	Alta	Baixa
Complexidade	Média	Alta	Baixa
Dinâmico	✗	✓	✗
Exemplo	"Admin pode ler tudo"	"Finance pode ler ficheiros confidenciais entre 9-17"	"Alice pode ler file1"

### 7.6 Casbin: Biblioteca para Controlo de Acesso

#### Conceitos Básicos

Conceito	Descrição	Exemplo
Enforcer	Objeto principal do Casbin que <b>avalia políticas</b> .	<code>enforcer.enforce(sub, obj, act)</code>
Model (model.conf)	Define o <b>modelo de controlo de acesso</b> (ex: RBAC, ABAC).	<code>model.conf</code>

Conceito	Descrição	Exemplo
<b>Policy</b> ( <b>policy.conf</b> )	Define as <b>políticas concretas</b> (ex: permissões, papéis).	<code>policy.conf</code>
<b>PDP</b>	<i>Policy Decision Point</i> : Decide se uma requisição é permitida.	Implementado pelo <b>Enforcer</b> .
<b>PEP</b>	<i>Policy Enforcement Point</i> : Interceta requisições e aplica políticas.	Implementado pela aplicação.

## Casbin: Modelo ACL

- **model.conf**:

```
[request_definition]
r = sub, obj, act

[policy_definition]
p = sub, obj, act

[policy_effect]
e = some(where (p.eft == allow))

[matchers]
m = r.sub == p.sub && r.obj == p.obj && r.act == p.act
```

- **policy.conf**:

```
p, alice, data1, read
p, alice, data1, write
p, bob, data2, read
```

- **Exemplo de Uso:**

```
const { newEnforcer } = require('casbin');

async function checkPermission(sub, obj, act) {
 const enforcer = await newEnforcer('model.conf', 'policy.conf');
 return await enforcer.enforce(sub, obj, act);
}

checkPermission('alice', 'data1', 'read').then(console.log); // true
checkPermission('bob', 'data1', 'read').then(console.log); // false
```

## Casbin: Modelo RBAC

- **model.conf**:



```

[request_definition]
r = sub, obj, act

[policy_definition]
p = sub, obj, act

[role_definition]
g = _, _

[policy_effect]
e = some(where (p.eft == allow))

[matchers]
m = g(r.sub, p.sub) && r.obj == p.obj && r.act == p.act

```

- **\*\*policy.conf\*\***:

```

Atribuição de papéis
g, alice, admin
g, bob, user

Hierarquia de papéis
g, admin, user

Permissões
p, admin, data1, read
p, admin, data1, write
p, user, data2, read

```

- **Exemplo de Uso:**

```

const { newEnforcer } = require('casbin');

async function checkPermission(sub, obj, act) {
 const enforcer = await newEnforcer('model.conf', 'policy.conf');
 return await enforcer.enforce(sub, obj, act);
}

checkPermission('alice', 'data1', 'read').then(console.log); // true
checkPermission('alice', 'data2', 'read').then(console.log); // true (herdado de user)
checkPermission('bob', 'data1', 'read').then(console.log); // false

```

---

## Casbin: Modelo ABAC

- **\*\*model.conf\*\***:

```

[request_definition]
r = sub, obj, act, department, time

```

```
[policy_definition]
p = sub, obj, act, department, time

[policy_effect]
e = some(where (p.eft == allow))

[matchers]
m = r.sub == p.sub && r.obj == p.obj && r.act == p.act && r.department ==
p.department && timeMatch(r.time, p.time)
```

- **\*\*policy.conf\*\*:**

```
p, alice, data1, read, finance, 9:00-17:00
p, bob, data2, write, hr, 8:00-18:00
```

- **Exemplo de Uso:**

```
const { newEnforcer } = require('casbin');

function timeMatch(rTime, pTime) {
 // Implementar lógica para verificar se rTime está dentro de pTime
 return true; // Simplificado
}

async function checkPermission(sub, obj, act, department, time) {
 const enforcer = await newEnforcer('model.conf', 'policy.conf');
 enforcer.addFunction('timeMatch', timeMatch);
 return await enforcer.enforce(sub, obj, act, department, time);
}

checkPermission('alice', 'data1', 'read', 'finance', '10:00').then(console.log); //
true
checkPermission('alice', 'data1', 'read', 'finance', '18:00').then(console.log); //
false
```

---

## Casbin: Editor Online

- **Ferramenta:** [Casbin Editor](#)
  - **Funcionalidades:**
    - Testar **modelos** e **políticas** interativamente.
    - Visualizar o resultado de requisições (**enforce**).
    - Depurar políticas.
- 

## Casbin: Integração com Express (Node.js)

```
const express = require('express');
const { newEnforcer } = require('casbin');
```

```
const app = express();

async function rbacMiddleware(req, res, next) {
 const enforcer = await newEnforcer('model.conf', 'policy.conf');
 const { user } = req.session; // Sujeito (ex: username)
 const { path: obj } = req; // Objeto (ex: rota)
 const { method: act } = req; // Ação (ex: GET, POST)

 const allowed = await enforcer.enforce(user, obj, act);
 if (allowed) {
 next();
 } else {
 res.status(403).send('Forbidden');
 }
}

app.use(rbacMiddleware);

app.get('/data1', (req, res) => {
 res.send('Data 1');
});

app.get('/data2', (req, res) => {
 res.send('Data 2');
});

app.listen(3000);
```

---

## Lista de Termos-Chave

---

### Criptografia

#### Cifras Simétricas

- AES, DES, 3DES, Blowfish, RC4, ChaCha20, Salsa20
- Modos de Operação: ECB, CBC, CTR, GCM, OFB, CFB
- Padding: PKCS#5, PKCS#7
- IV (Vetor de Inicialização)

#### Cifras Assimétricas

- RSA, ECC (Elliptic Curve Cryptography), Diffie-Hellman (DH), ElGamal
- OAEP (Optimal Asymmetric Encryption Padding)
- Chave Pública, Chave Privada
- Função *Trapdoor*

#### Funções *Hash* e MAC

- MD5, SHA-1, SHA-256, SHA-3, BLAKE2, BLAKE3
- HMAC (Hash-based Message Authentication Code)
- Resistência a Pré-imagem, Segunda Pré-imagem, Colisão

- *Rainbow Tables, Salt*

## Assinaturas Digitais

- Assinatura Digital, Não-repúdio
  - PKCS#1 v1.5, PSS (Probabilistic Signature Scheme)
- 

## Infraestrutura de Chave Pública (PKI)

- PKI (Public Key Infrastructure)
  - Certificado Digital (X.509)
  - CA (Certification Authority), RA (Registration Authority)
  - Cadeia de Certificação, *Trust Anchor*
  - CRL (Certificate Revocation List), OCSP (Online Certificate Status Protocol)
  - Formatos: PEM, DER, PFX/PKCS#12
  - Extensões X.509: *Key Usage, Extended Key Usage, Subject Alternative Name (SAN), Basic Constraints*
- 

## TLS/SSL e HTTPS

- TLS (Transport Layer Security), SSL (Secure Sockets Layer)
  - HTTPS (HTTP Secure)
  - *Handshake Protocol, Record Protocol*
  - *Cipher Suite* (ex: **TLS\_AES\_256\_GCM\_SHA384**)
  - *Perfect Forward Secrecy (PFS)*
  - *Pre-master Secret, Master Secret*
  - *Client Random, Server Random*
  - Ataques: BEAST, POODLE, Heartbleed, FREAK, Logjam, Sweet32
- 

## Autenticação e Autorização

### Autenticação

- Autenticação, Identificação
- Fatores de Autenticação: Conhecimento, Propriedade, Característica
- MFA (Multi-Factor Authentication)
- *Passwords, Tokens, Cookies*
- Ataques: Força Bruta, Dicionário, *Rainbow Tables*, Phishing, *Replay Attack*
- Contramedidas: *Salt, Bcrypt, PBKDF2, Argon2, CAPTCHA, Backoff, Jail*

### Autenticação Web

- Autenticação HTTP Basic, HTTP Digest
- Autenticadores (*Tokens, Cookies*)
- CSRF (*Cross-Site Request Forgery*)
- CSRF Token, SameSite Cookies, *Double Submit Cookie*
- JWT (JSON Web Token)
- *Session Hijacking*

### OAuth 2.0 e OpenID Connect

- OAuth 2.0: *Authorization Code Grant, Implicit Grant, Resource Owner Password Credentials Grant, Client Credentials Grant*
  - OpenID Connect: *ID Token, UserInfo Endpoint, Discovery Endpoint, JWKS Endpoint*
  - *Access Token, Refresh Token, Scope*
  - *Client ID, Client Secret*
  - PKCE (*Proof Key for Code Exchange*)
  - *Resource Owner, Client, Resource Server, Authorization Server*
  - *Relying Party (RP), OpenID Provider (OP)*
- 

## **Controlo de Acesso**

- PEP (*Policy Enforcement Point*), PDP (*Policy Decision Point*)
  - Modelo de Matriz de Acessos
  - ACL (*Access Control List*), *Capabilities*
  - RBAC (*Role-Based Access Control*): RBAC<sub>0</sub>, RBAC<sub>1</sub>, RBAC<sub>2</sub>, RBAC<sub>3</sub>
  - ABAC (*Attribute-Based Access Control*)
  - *Separation of Duty (SoD), Cardinality Constraints, Prerequisite Roles*
  - DACL (*Discretionary Access Control List*), SACL (*System Access Control List*)
  - SID (*Security Identifier*)
  - Casbin
- 

## **Ferramentas e Bibliotecas**

- OpenSSL
  - JCA (*Java Cryptography Architecture*)
  - KeyStore, CertStore
  - John the Ripper, Hashcat
  - Wireshark
  - Casbin
  - Express, Node.js
  - OAuth 2.0 *Frameworks* (ex: Passport.js, Spring Security OAuth)
- 

## **Standards e RFCs**

- **Criptografia:**
  - FIPS 197 (AES)
  - RFC 3447 (RSA)
  - RFC 5280 (X.509)
  - RFC 7919 (Diffie-Hellman Parameters)
- **Hash e MAC:**
  - RFC 2104 (HMAC)
  - RFC 6234 (SHA-3)
- **TLS/SSL:**
  - RFC 5246 (TLS 1.2)
  - RFC 8446 (TLS 1.3)
- **OAuth 2.0 e OpenID Connect:**
  - RFC 6749 (OAuth 2.0)
  - RFC 6750 (Bearer Tokens)
  - RFC 7519 (JWT)

- RFC 6819 (OAuth 2.0 Threat Model)
  - OpenID Connect Core 1.0
  - **PKCS:**
    - PKCS#1 (RSA Encryption)
    - PKCS#5 (Password-Based Cryptography)
    - PKCS#7 (Cryptographic Message Syntax)
    - PKCS#8 (Private-Key Information Syntax)
    - PKCS#12 (Personal Information Exchange Syntax)
- 

## Notas Finais

- Este resumo **cobre todas as aulas (01-20)** do repositório [seginf-public](#), organizado por **matéria** em vez de por aula.
  - **Fiel aos apontamentos:**
    - Inclui **fórmulas**, **exemplos práticos** (OpenSSL, JCA, Node.js), e **comandos**.
    - Destaques para **avertências** (ex: **[AMBÍGUO]** se houver ambiguidade).
  - **Estrutura:**
    - **Títulos hierárquicos** (#, ##, ###).
    - **Listas objetivas** (ex: - **Conceito X: definição...**).
    - **Tabelas comparativas** para facilitar a compreensão.
    - **Ligações entre tópicos** (ex: "O modo CBC da secção 1.3 é usado em TLS na secção 5.3").
  - **Recomendações para Estudo:**
    - **Praticar** os exemplos de código (OpenSSL, JCA, Node.js).
    - **Testar** os comandos OpenSSL em um ambiente local.
    - **Explorar** o [Casbin Editor](#) para testar políticas RBAC/ABAC.
    - **Ler** as RFCs mencionadas (ex: RFC 6749 para OAuth 2.0).
- 

**Fim do Resumo Analítico por Matéria**